

Chapter 1 : Introduction to Spring MVC

1. Introduction

Spring MVC is a **web framework** provided by the Spring Framework for developing **Java web applications** using the **Model-View-Controller (MVC)** design pattern.

It separates the application into three independent parts:

- Model → Business Logic + Data
- View → User Interface (JSP/Thymeleaf)
- Controller → Request Handler

Instead of writing large Servlets, Spring MVC provides annotations and dependency injection to make development easier, cleaner, and more maintainable.

Spring MVC is built **on top of the Servlet API**. At its core, it still uses a servlet called **DispatcherServlet**, but developers rarely interact with raw servlet code.

2. Why Spring MVC Was Needed

Before Spring MVC, developers primarily built web applications using:

- Servlets
- JSP
- JDBC
- Manual URL mapping

A typical Servlet project looked like this:

User Request

↓

Servlet

↓

Business Logic

↓

Database



Servlet



JSP

Problems:

- Huge Servlet classes (1000+ lines)
- HTML mixed with Java code
- Manual request processing
- Difficult testing
- Poor maintainability
- Tight coupling
- Code duplication

As projects grew, maintaining them became increasingly difficult.

Spring MVC solved these issues by promoting:

- Separation of concerns
- Annotation-based programming
- Loose coupling
- Better testing
- Cleaner architecture

3. Historical Background

Phase 1 (1997)

CGI Programs

Problems:

- Slow

- One process per request
-

Phase 2

Servlets

Advantages:

- Faster than CGI
- Java-based

Problems:

- Huge classes
 - Mixed responsibilities
-

Phase 3

JSP

Advantages:

- Easier UI development

Problems:

- Business logic often ended up in JSP pages
-

Phase 4

MVC Frameworks

Frameworks introduced:

- Struts
- JSF
- Spring MVC

These frameworks enforced proper separation of concerns.

Phase 5

Spring MVC (2003)

Rod Johnson introduced Spring Framework.

Spring MVC became popular because of:

- Dependency Injection

- IoC
 - Annotation support
 - Easy testing
 - Flexible architecture
-

Phase 6

Spring Boot + Spring MVC

Today, most enterprise Java web applications use:

- Spring Boot
 - Spring MVC
 - Spring Data JPA
 - Spring Security
 - REST APIs
-

4. Real-World Analogy

Imagine a restaurant.

Customer

Places an order.

↓

Waiter

Takes the order and forwards it to the kitchen.

↓

Kitchen

Prepares the food.

↓

Waiter

Brings the food back.

↓

Customer

Receives the food.

Now map this to Spring MVC:

Restaurant Spring MVC

Customer Browser

Waiter Controller

Kitchen Service Layer

Store Room Database

Plate Model

Dining Table View

The waiter never cooks the food.

Similarly, the Controller should never contain business logic.

5. What is MVC?

MVC stands for:

- Model
- View
- Controller

It divides responsibilities into separate layers.

A. Model

The Model contains:

- Business logic
- Data
- Database operations
- Validation rules

Example:

Employee

↓

EmployeeService



EmployeeRepository



Database

Responsibilities:

- Store data
- Retrieve data
- Process data

The Model never displays anything.

B. View

The View is responsible only for displaying information.

Examples:

- JSP
- Thymeleaf
- HTML

Example:

Welcome John

Employee List

Product Details

A View should not contain business logic.

C. Controller

The Controller acts as the middleman.

Responsibilities:

- Receive HTTP request

- Read request parameters
- Call service layer
- Receive data
- Send data to View

Example:

```
@GetMapping("/home")  
public String home() {  
    return "home";  
}
```

6. Spring MVC Architecture

Browser

|

HTTP Request

|



DispatcherServlet

|



Controller

|



Service Layer



Repository Layer



Database



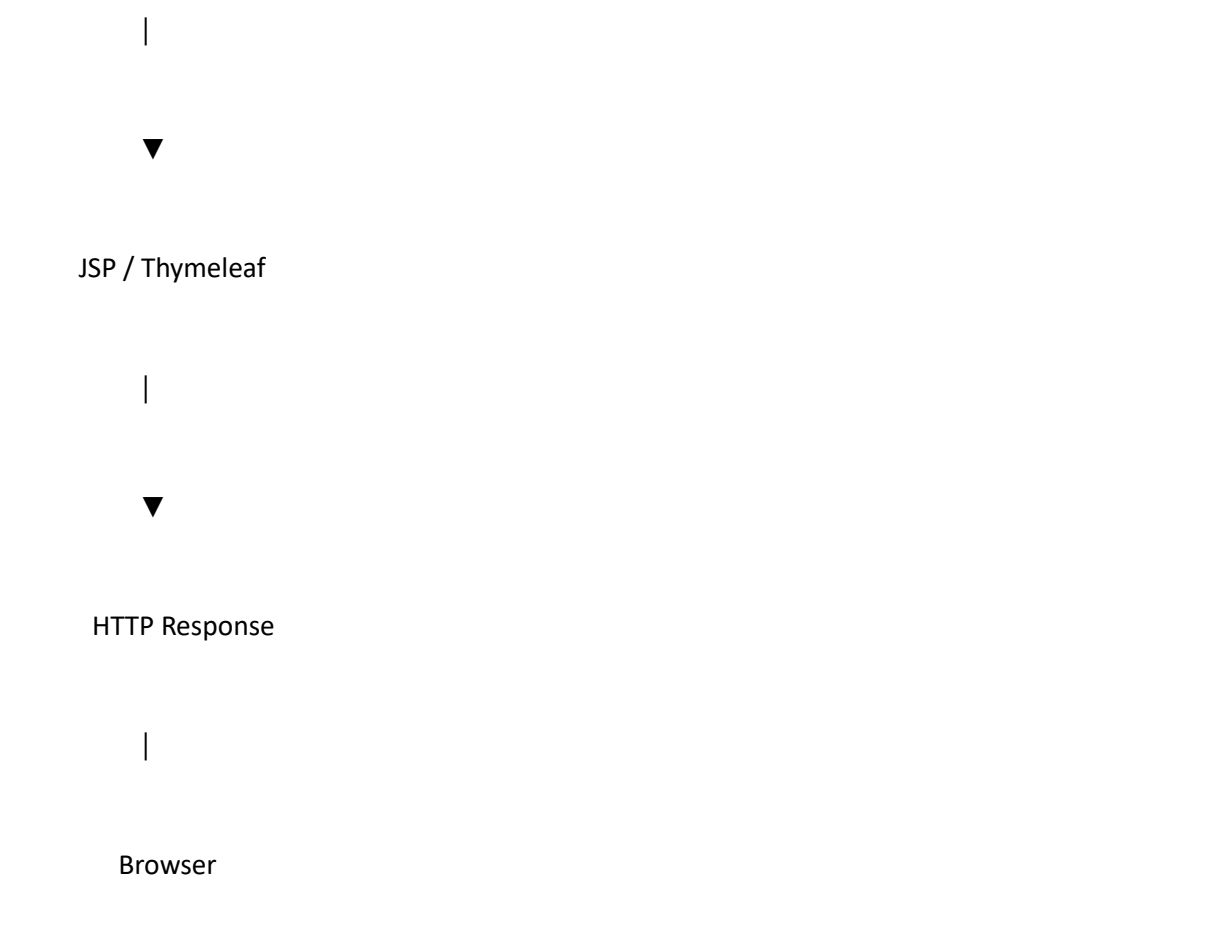
Result/Data



Controller



View Resolver



7. Internal Working (High-Level)

Suppose the user visits:

`http://localhost:8080/home`

Step 1

Browser sends request.



Step 2

DispatcherServlet receives it.



Step 3

Looks for matching Controller.



Step 4

Controller method executes.

↓

Step 5

Business logic executes.

↓

Step 6

Data returned.

↓

Step 7

Controller returns view name.

↓

Step 8

ViewResolver finds JSP/Thymeleaf.

↓

Step 9

HTML is generated.

↓

Step 10

Browser displays the page.

8. Syntax (Basic Controller)

@Controller

```
public class HomeController {
```

```
    @GetMapping("/home")
```

```
    public String home() {
```

```
        return "home";
```

```
    }
```

```
}
```

Explanation:

- @Controller → Marks the class as a Spring MVC controller.
- @GetMapping("/home") → Maps GET requests to /home.

- return "home" → Returns the logical view name. The ViewResolver resolves it to the actual page (e.g., home.jsp or a Thymeleaf template).
-

9. Execution Flow

User types:

localhost:8080/home

↓

HTTP Request

↓

DispatcherServlet

↓

HomeController

↓

home()

↓

returns "home"

↓

ViewResolver

↓

home.jsp

↓

HTML Response

↓

Browser

10. Industry Usage

Spring MVC is widely used in:

- Banking applications
- E-commerce platforms
- Hospital Management Systems
- ERP software

- CRM applications
- Government portals
- Insurance systems
- Educational platforms
- HR Management Systems
- Enterprise Admin Dashboards

It is commonly combined with:

- Spring Boot
 - Spring Data JPA
 - Hibernate
 - Spring Security
 - REST APIs
 - Thymeleaf
 - JSP (legacy projects)
-

11. Advantages

- Clear separation of concerns.
 - Annotation-based configuration.
 - Easy integration with Spring ecosystem.
 - Highly testable.
 - Supports multiple view technologies (JSP, Thymeleaf, etc.).
 - Flexible URL mapping.
 - Simplifies form handling and validation.
 - Scales well for enterprise applications.
-

12. Disadvantages

- Initial learning curve for beginners.
 - More configuration than minimal frameworks (though Spring Boot reduces this).
 - Can be overkill for very small web applications.
 - Requires understanding of the Spring container and MVC lifecycle.
-

13. When to Use

Use Spring MVC when:

- Building enterprise web applications.
 - Developing applications with complex business logic.
 - You need clean architecture and maintainable code.
 - You want to leverage the broader Spring ecosystem.
-

14. When NOT to Use

Avoid Spring MVC when:

- Building a very small static website.
 - A lightweight framework or simple servlet is sufficient.
 - The application does not require server-side rendering (consider REST APIs with a JavaScript frontend instead).
-

15. Best Practices

- Keep controllers thin; move business logic to service classes.
 - Use meaningful URL mappings.
 - Validate user input.
 - Handle exceptions centrally.
 - Follow the MVC separation strictly.
 - Prefer constructor injection for dependencies.
 - Keep views free from business logic.
-

16. Common Mistakes

- Putting database logic inside controllers.
 - Writing business logic in JSP pages.
 - Returning physical view paths instead of logical names.
 - Ignoring validation.
 - Using one controller for unrelated modules.
 - Catching every exception inside controllers instead of using global exception handling.
-

17. Beginner Example

@Controller

```
public class WelcomeController {
```

```
    @GetMapping("/welcome")
```

```
    public String welcome() {
```

```
        return "welcome";
```

```
    }
```

```
}
```

Request:

GET /welcome

Returned View:

welcome.jsp

18. Intermediate Example

@Controller

```
public class EmployeeController {
```

```
    @GetMapping("/employee")
```

```
    public String employee(Model model) {
```

```
        model.addAttribute("name", "John");
```

```
        return "employee";
```

```
    }
```

```
}
```

The controller adds data to the Model, and the view (employee.jsp or Thymeleaf template) displays it.

19. Industry-Level Example

@Controller

```

@RequestMapping("/employees")

public class EmployeeController {

    private final EmployeeService service;

    public EmployeeController(EmployeeService service) {
        this.service = service;
    }

    @GetMapping
    public String listEmployees(Model model) {

        model.addAttribute("employees",
            service.getAllEmployees());

        return "employee-list";
    }
}

```

Here, the controller delegates all business work to the EmployeeService, keeping the controller focused only on handling requests and preparing the view model.

Topic 2: DispatcherServlet (Front Controller Concept)

This is **the most important topic in Spring MVC**.

If you thoroughly understand **DispatcherServlet**, you'll understand **how every request travels through Spring MVC**.

Nearly every Spring MVC interview includes questions about DispatcherServlet and the **Front Controller Pattern**.

1. Introduction

DispatcherServlet is the **heart (brain) of Spring MVC**.

It is the **first component** that receives every incoming HTTP request and the **last component** that sends the HTTP response back to the client.

Think of it as a **traffic controller**.

Instead of allowing every URL to directly access controllers, **all requests first go through DispatcherServlet.**

Browser



DispatcherServlet



Controller



Business Logic



View



DispatcherServlet



Browser

Without DispatcherServlet, Spring MVC cannot function.

2. Why DispatcherServlet is Needed

Imagine a city without a traffic police officer.

Car A

Car B

Car C

Car D

↓

Road

↓

Accidents

Confusion

Wrong Directions

Now imagine a traffic officer directing every vehicle.

Car

↓

Traffic Police

↓

Correct Road

↓

Destination

DispatcherServlet plays exactly the same role.

It decides

- Which Controller should execute

- Which method should execute
- Which View should open
- Which Model should be sent
- Which Exception Handler should execute

Without it, developers would need to manually:

- Parse URLs
- Match controllers
- Call methods
- Render JSPs
- Handle errors
- Manage request forwarding

Spring MVC automates all of this.

3. Historical Background

Before Spring MVC, Java developers mostly used Servlets.

Suppose a website had

- Login
- Register
- Profile
- Products
- Orders
- Payments

Developers often created separate servlets.

LoginServlet

RegisterServlet

ProductServlet

OrderServlet

PaymentServlet

Every servlet had code like

```
if(request.getParameter("action").equals("login"))
```

or

```
switch(action)
```

As applications grew

- duplicate code increased
- maintenance became difficult
- request routing became messy

To solve this, enterprise frameworks adopted the **Front Controller Pattern**.

Spring MVC implemented this pattern using a single servlet:

DispatcherServlet

4. Real-World Analogy

Imagine a hospital.

Patients do not directly go to doctors.

Instead,

Patient

↓

Reception

↓

Correct Doctor

↓

Treatment



Reception



Patient Leaves

Who decides which doctor?

Reception.

DispatcherServlet is the reception.

It receives every request.

Then decides

- Login Controller
- Product Controller
- Employee Controller
- Admin Controller

5. What is Front Controller Pattern?

The Front Controller Pattern is a design pattern in which

One central controller receives all client requests and dispatches them to appropriate handlers.

Instead of

Browser



100 Controllers

We have

Browser



DispatcherServlet



Controller

Only one servlet handles everything.

Why is it called "Front" Controller?

Because it stands **at the front** of the application.

Nothing enters Spring MVC without passing through it.

Traditional Servlet Architecture

Browser



LoginServlet



Response

Browser



ProductServlet



Response

Browser



RegisterServlet



Response

Every servlet receives requests independently.

Spring MVC Architecture

Browser



DispatcherServlet



Handler Mapping



Controller



Service



Repository



Database



View Resolver



JSP



Browser

Everything is centralized.

6. Internal Working of DispatcherServlet

This is the most important interview question.

Suppose the user types

`http://localhost:8080/student/home`

Let's understand what happens internally.

Step 1

Browser sends HTTP Request.

`GET /student/home`



DispatcherServlet receives it.

Step 2

DispatcherServlet asks

Which Controller handles

/student/home ?

It consults

HandlerMapping

Step 3

HandlerMapping searches

@Controller

@RequestMapping

@GetMapping

Suppose it finds

@GetMapping("/student/home")

public String home()

It returns

HomeController.home()

Step 4

DispatcherServlet now asks

How do I invoke this method?

It contacts

HandlerAdapter

Step 5

HandlerAdapter calls

home();

Controller executes.

Step 6

Controller calls

Service Layer

Example

```
studentService.getStudents();
```

Step 7

Service calls Repository.

Repository fetches data from database.

Database

↓

Student List

Step 8

Repository returns

↓

Service

↓

Controller

Step 9

Controller prepares Model.

```
model.addAttribute("students",list);
```

Then returns

```
return "student";
```

Notice

student

is NOT a JSP.

It is just a logical view name.

Step 10

DispatcherServlet receives

student

It now asks

ViewResolver

Where is "student"?

Step 11

ViewResolver converts

student

into

/WEB-INF/views/student.jsp

Step 12

JSP renders HTML.

Step 13

DispatcherServlet sends HTML response.

Browser displays page.

Complete Internal Flow Diagram

Browser

↓

HTTP Request

↓

DispatcherServlet



HandlerMapping



Controller Found



HandlerAdapter



Controller Method



Service



Repository



Database



Repository



Service



Controller



Model



Logical View Name



DispatcherServlet



ViewResolver



JSP



HTML



DispatcherServlet



Browser

This complete cycle happens for **every request**.

7. Important Components Used by DispatcherServlet

1. HandlerMapping

Purpose

Finds the correct controller.

Example

URL



HandlerMapping



Controller

Without HandlerMapping,

DispatcherServlet would never know which controller to execute.

2. HandlerAdapter

Purpose

Invokes controller methods.

Why?

Different controllers may have different method signatures.

Example

```
public String home()
```

or

```
public String login(Model model)
```

or

```
public String save(Employee emp)
```

HandlerAdapter knows how to invoke each method correctly.

3. ViewResolver

Purpose

Finds the view.

Example

Controller returns

```
return "home";
```

ViewResolver converts it to

```
/WEB-INF/views/home.jsp
```

4. Model

Stores data.

Example

```
model.addAttribute("name","John");
```

Data travels to JSP through Model.

5. View

Displays data.

Examples

- JSP
- Thymeleaf

8. DispatcherServlet Lifecycle

Like every servlet,

DispatcherServlet also follows a lifecycle.

Load



Instantiate



Initialize



Receive Requests



Process Requests



Destroy

Internally

init()



service()

↓

destroy()

Exactly like normal Servlets.

Difference:

Spring performs most lifecycle management automatically.

9. Syntax (web.xml Configuration - Traditional)

Before Spring Boot, DispatcherServlet was configured manually.

```
<servlet>
```

```
    <servlet-name>spring</servlet-name>
```

```
    <servlet-class>
```

```
        org.springframework.web.servlet.DispatcherServlet
```

```
    </servlet-class>
```

```
    <load-on-startup>1</load-on-startup>
```

```
</servlet>
```

```
<servlet-mapping>
```

```
    <servlet-name>spring</servlet-name>
```

```
    <url-pattern>/</url-pattern>
```

```
</servlet-mapping>
```

Explanation

`<servlet-class>`

`org.springframework.web.servlet.DispatcherServlet`

Creates DispatcherServlet.

`<url-pattern>/</url-pattern>`

Means

Every request

`/home`

`/login`

`/register`

`/products`

will first go to DispatcherServlet.

`<load-on-startup>1</load-on-startup>`

Creates DispatcherServlet when server starts instead of waiting for the first request.

10. Spring Boot Configuration

In Spring Boot, **you don't configure DispatcherServlet manually.**

Spring Boot automatically creates and registers it through **auto-configuration.**

You simply write controllers.

`@Controller`

`public class HomeController {`

`@GetMapping("/")`

`public String home() {`

`return "home";`

`}`

}

No XML required.

11. Execution Flow

Suppose

localhost:8080/login

Execution

Browser

↓

DispatcherServlet

↓

HandlerMapping

↓

LoginController

↓

login()

↓

Service

↓

Database



Model



ViewResolver



login.jsp



Browser

Topic 3: Controllers in Spring MVC (@Controller, @RequestMapping, @GetMapping, @PostMapping)

This is one of the most important topics in Spring MVC.

Every request that reaches **DispatcherServlet** is ultimately handled by a **Controller**.

Controllers act as the **entry point** of your application's business logic.

1. Introduction

A **Controller** is a Java class that receives HTTP requests from the client, processes them (usually by calling the Service layer), and returns a response.

In Spring MVC, a controller is identified using the `@Controller` annotation.

Example:

`@Controller`

```
public class HomeController {
```

```
    @GetMapping("/home")
```

```
public String home() {  
    return "home";  
}  
}
```

When the user visits:

`http://localhost:8080/home`

The request flows like this:

Browser

|



DispatcherServlet

|



HomeController

|



home()

|



home.jsp / Thymeleaf

2. Why Controllers are Needed

Imagine a customer calling a company's customer support.

The customer does **not** directly contact:

- Database
- Accountant
- Warehouse
- Delivery Team

Instead, they first speak to a **Customer Support Executive**, who forwards the request to the appropriate department.

Similarly,

The browser never directly communicates with:

- Service Layer
- Repository Layer
- Database

Instead,

Browser



Controller



Service



Repository



Database

The Controller coordinates the process.

3. Historical Background

Before Spring MVC

Developers used Servlets.

Example:

```
@WebServlet("/login")
```

```
public class LoginServlet extends HttpServlet{
```

```
protected void doGet(HttpServletRequest request,
                    HttpServletResponse response){

}

}
```

Problems:

- Large servlet classes
 - Manual request parsing
 - Difficult testing
 - Tight coupling
 - Repetitive code
-

Spring MVC introduced annotation-based controllers.

Instead of writing:

```
public class LoginServlet
extends HttpServlet
```

we simply write:

```
@Controller
public class LoginController{

}
```

Much simpler.

4. Real-World Analogy

Think about a hospital.

Patient

↓

Reception Desk

↓

Doctor

↓

Lab

↓

Reception

↓

Patient

The Reception Desk:

- Receives patient
- Finds doctor
- Sends reports
- Guides patient

Controller behaves exactly like Reception.

5. Internal Working

Suppose user enters

`http://localhost:8080/student/list`

Internal Flow

Browser

↓

DispatcherServlet



HandlerMapping



StudentController



listStudents()



StudentService



Repository



Database



Model



ViewResolver



student.jsp

Controller is responsible for

- Receiving request
- Reading parameters
- Calling Service
- Sending Model
- Returning View

6. What is @Controller?

@Controller is a Spring stereotype annotation.

It tells the Spring IoC Container:

"This class is a Spring MVC Controller."

Example

@Controller

```
public class HomeController{
```

```
}
```

Without @Controller

Spring will ignore the class.

Internal Working

During application startup,

Component Scanning begins.

Spring scans packages.

Package

↓

Class

↓

@Controller Found

↓

Object Created

↓

Bean Stored

The object is stored inside the Spring IoC Container.

Syntax

@Controller

```
public class HomeController{
```

```
}
```

Example

@Controller

```
public class WelcomeController{
```

```
    @GetMapping("/welcome")
```

```
    public String welcome(){
```

```
        return "welcome";
```

```
}
```

```
}
```

7. What is @RequestMapping?

@RequestMapping maps a URL to a controller or a specific method.

It can be used:

- At Class Level
- At Method Level

Syntax (Method Level)

```
@RequestMapping("/home")
```

```
public String home(){
```

```
}
```

Syntax (Class Level)

```
@Controller
```

```
@RequestMapping("/student")
```

```
public class StudentController{
```

```
}
```

Now all methods begin with

```
/student
```

Example

```
@GetMapping("/add")
```

Final URL

```
/student/add
```

Internal Working

Suppose

http://localhost:8080/student/add

HandlerMapping searches

StudentController

↓

@RequestMapping("/student")

+

@GetMapping("/add")

↓

Combined URL

↓

/student/add

Example

@Controller

@RequestMapping("/employee")

public class EmployeeController{

 @GetMapping("/list")

 public String list(){

 return "employee";

```
}
```

```
}
```

URL

localhost:8080/employee/list

Attributes of @RequestMapping

Most commonly used attributes:

value

```
@RequestMapping("/home")
```

method

```
@RequestMapping(  
value="/save",  
method=RequestMethod.POST)
```

produces

Defines response type.

```
produces="application/json"
```

consumes

Defines accepted request type.

```
consumes="application/json"
```

8. What is @GetMapping?

Introduced in Spring 4.3.

Shortcut for

```
@RequestMapping(  
method=RequestMethod.GET)
```

Used for

- Reading data

- Displaying pages
 - Fetching records
-

Syntax

```
@GetMapping("/home")
```

Equivalent to

```
@RequestMapping(  
value="/home",  
method=RequestMethod.GET)
```

Example

```
@Controller  
  
public class HomeController{  
  
    @GetMapping("/home")  
    public String home(){  
  
        return "home";  
  
    }  
  
}
```

Internal Working

Browser

GET /home

↓

DispatcherServlet

↓

Controller

↓

home()

↓

home.jsp

GET Request Characteristics

- ✓ Safe
 - ✓ Idempotent
 - ✓ No database modification
 - ✓ Used for fetching data
-

Example

GET

/products

/students

/home

/login

9. What is @PostMapping?

Shortcut for

@RequestMapping(

method=RequestMethod.POST)

Used for

- Saving data
- Registration
- Login
- Updating
- Uploading files

Syntax

```
@PostMapping("/save")
```

Example

```
@PostMapping("/save")
```

```
public String save(){
```

```
    return "success";
```

```
}
```

Internal Flow

Browser

↓

POST /save

↓

DispatcherServlet

↓

Controller

↓

Service

↓

Database

↓

success.jsp

POST changes data.

Examples

Add Student

Register User

Book Ticket

Make Payment

Upload Resume

GET vs POST

Feature	GET	POST
Purpose	Read Data	Save Data
Data in URL	Yes	No (Request Body/Form Data)
Bookmark Possible	Yes	No
Secure	Less	More (still use HTTPS for security)
Used For	Search, View Registration, Login, Insert	

10. Execution Flow

Suppose

localhost:8080/student/save

Browser



DispatcherServlet



HandlerMapping



StudentController



saveStudent()



StudentService



Repository



Database

↓

return "success"

↓

ViewResolver

↓

success.jsp

↓

Browser

11. Industry-Level Folder Structure

src

└─ main

│ └─ java

│ └─ controller

│ └─ service

│ └─ repository

│ └─ entity

| └─ config

└─ webapp

└─ WEB-INF

└─ views

Controllers stay inside
controller
package.

12. Industry Usage

Controllers are used in almost every enterprise module:

Banking

AccountController

LoanController

CustomerController

Hospital

DoctorController

PatientController

AppointmentController

E-Commerce

ProductController

CartController

OrderController

PaymentController

University

StudentController

FacultyController

CourseController